

CO4-AT1

Name: R. Usha

Reg no: 192525100

1) A system needs to store large data and perform frequent search operations. For this, compare Hashing and Sorting-based search (Binary Search) in terms of:

- a) Time complexity
- b) Space usage
- c) Update operations
- d) Real-world applicability

Conclude which is better for dynamic systems.

Hashing vs. Sorting-Based Search (Binary Search)

For a system that stores a large amount of data and performs frequent search operations, **Hashing** and **Binary Search** are two important techniques. Their suitability depends on search speed, memory availability, and how frequently the data is updated.

Factor Hashing Binary Search

a) Time Complexity	Average search: $O(1)$; Worst case: $O(n)$ due to collisions	Search: $O(\log n)$, but data must be sorted
b) Space Usage	Generally requires $O(n)$ extra space for the hash table	Requires $O(1)$ extra space for iterative binary search
c) Update Operations	Insert/Delete: $O(1)$ average, making updates efficient	Insertion/Deletion can be $O(n)$ in an array because elements may need shifting
d) Real-World Applicability	Databases, caches, symbol tables, dictionaries, password lookup, indexing	Searching sorted arrays, databases, dictionaries.

Analysis

Hashing:

Hashing converts a key into an index using a hash function. This allows direct access to the

required element. Under normal conditions, search, insertion, and deletion take **$O(1)$ average time**. However, collisions may occur when multiple keys map to the same location, potentially increasing the search time.

Binary Search:

Binary Search works on a **sorted dataset**. It repeatedly divides the search space into two halves, resulting in **$O(\log n)$** search complexity. However, maintaining sorted data becomes costly when frequent insertions and deletions are required.

Conclusion

For a **dynamic system with frequent search, insertion, and deletion operations**, **Hashing is generally better** because it provides average **$O(1)$** search and update operations. Binary Search is preferable when the data is relatively static, already sorted, and memory efficiency is important.

Example: An online shopping system can use hashing to quickly locate products using product IDs, while Binary Search is more suitable for searching a fixed, sorted list of product prices.

2) Compare Quick Sort and Merge Sort based on:

- a) Best, average, and worst-case complexity
- b) Stability
- c) Memory usage
- d) Practical performance

Explain why Quick Sort is often preferred in real applications despite its worst-case complexity.

Quick Sort vs. Merge Sort

Quick Sort and Merge Sort are efficient comparison-based sorting algorithms, but they differ significantly in complexity, stability, memory requirements, and practical performance.

Factor	Quick Sort	Merge Sort
a) Best Case	$O(n \log n)$	$O(n \log n)$
Average Case	$O(n \log n)$	$O(n \log n)$
Worst Case	$O(n^2)$	$O(n \log n)$
b) Stability	Not stable in its standard implementation	Stable
c) Memory Usage	Usually $O(\log n)$ auxiliary space for recursion	Requires $O(n)$ auxiliary space for arrays
d) Practical Performance	Usually very fast due to good cache locality and low overhead	Predictable performance, especially good for large/external datasets

Analysis

a) Time Complexity

Quick Sort:

- Best Case: **$O(n \log n)$** – pivot divides the array into nearly equal parts.
- Average Case: **$O(n \log n)$** .
- Worst Case: **$O(n^2)$** – occurs when the pivot repeatedly produces highly unbalanced partitions.
- Choosing a good pivot, such as a **random pivot or median-of-three**, greatly reduces the probability of the worst case.

Merge Sort:

- Best Case: **$O(n \log n)$**
- Average Case: **$O(n \log n)$**
- Worst Case: **$O(n \log n)$**

Thus, Merge Sort provides a guaranteed upper bound, whereas Quick Sort can have a quadratic worst case.

b) Stability

Merge Sort is stable, meaning elements having equal keys retain their original relative order.

Standard **Quick Sort is not stable**, because partitioning can change the relative positions of equal elements.

c) Memory Usage

Quick Sort is generally considered more **memory-efficient** because it sorts largely in place. Its recursive implementation typically requires **$O(\log n)$** auxiliary stack space on average.

Merge Sort generally requires **$O(n)$** additional memory for merging when implemented on arrays.

d) Practical Performance

Although Merge Sort has a guaranteed **$O(n \log n)$** worst-case complexity, Quick Sort is often faster in practice. This is mainly because:

1. **Better cache locality:** Quick Sort works mostly within the same array, making good use of CPU cache.
2. **In-place operation:** It generally does not require a separate array of size n .
3. **Lower memory overhead:** Less additional memory is required.
4. **Small constant factors:** Its operations are often simple and efficient.
5. **Good pivot selection:** Randomized or median-based pivot selection reduces the chance of highly unbalanced partitions.

Why is Quick Sort often preferred?

Quick Sort demonstrates an important principle in algorithm design: **asymptotic worst-case complexity does not always determine practical performance**. Although its worst case is **$O(n^2)$** , a well-implemented Quick Sort with good pivot selection performs **$O(n \log n)$ on average** and often outperforms Merge Sort for in-memory array sorting.

Final Conclusion

Quick Sort is often preferred for **in-memory sorting** because of its speed, low memory overhead, and excellent cache performance. **Merge Sort** is preferred when **stable sorting, predictable $O(n \log n)$ performance, or external/large-scale sorting** is required.

Assessment Point:

Quick Sort → Faster and memory-efficient in many practical cases

Merge Sort → Stable and guarantees **$O(n \log n)$ worst-case performance**